

NPM & Modern JavaScript Tooling — প্রজেক্ট আর্কিটেকচার ও প্রোডাকশন পাইপলাইন

বাস্তব দুনিয়ায় প্রফেশনাল জাভাস্ক্রিপ্ট অ্যাপ্লিকেশন কখনো শূন্য থেকে একা তৈরি করা হয় না। বিশ্বজুড়ে লক্ষ লক্ষ ডেভেলপারের তৈরি প্যাকেজ নিরাপদে ম্যানেজ করা, নির্ভুল ডিপেনডেন্সি লক রাখা এবং স্বয়ংক্রিয় বিল্ড ও লিন্টিং পাইপলাইন পরিচালনা করা হলে আধুনিক সফটওয়্যার ইঞ্জিনিয়ারিংয়ের মূল ভিত্তি। এই চ্যাপ্টারে আমরা **package.json**, **Semantic Versioning (SemVer)**, **Lockfiles**, **npm vs npx**, **Vite**, **Bundlers**, **Tree Shaking**, **ESLint**, **Prettier**, **.env** সিকিউরিটি এবং CI/CD প্রোডাকশন ডিপ্লয়মেন্ট শিখব।

 1. package.json & Node 2. SemVer Architecture 3. Lockfile & npm ci 4. Dependencies vs Dev 5. npm scripts & npx 6. Vite & Bundling 7. Tree Shaking & Maps 8. ESLint & Prettier 9. .env & Security Audit 10. CI/CD Deployment

Part 01 — Package Concepts, Node Ecosystem & Project Initialization (34.1 – 34.5)

34.1 Package কী?

Package হলো reusable code-এর একটি collection, যেটা অন্য developer নিজের project-এ ব্যবহার করতে পারে। ধরো তুমি নিজে একটা function লিখলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

কিন্তু একজন developer এমন একটি library তৈরি করল যেখানে হাজার হাজার useful function আছে। তুমি সেই library install করে ব্যবহার করতে পারো।

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Your Project

- Your Code
- External Packages
 - React
 - Express
 - Axios
 - Lodash
 - etc.

34.2 NPM কী?

NPM = Node Package Manager এটি JavaScript ecosystem-এর সবচেয়ে পরিচিত package manager-গুলোর একটি। NPM দিয়ে আমরা:

- package install
- package uninstall
- package update
- dependency manage
- scripts run
- project initialize

ইত্যাদি করতে পারি।

♦ Example

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install lodash
```

এখানে:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
npm
↓
install
↓
lodash
```

34.3 Node.js এবং NPM-এর সম্পর্ক

Node.js install করলে সাধারণত npm-ও পাওয়া যায়। Check করতে:

TERMINAL COMMAND

BASH

```
-weight:700;">node -v
```

TERMINAL COMMAND

BASH

```
-weight:700;">npm -v
```

Example:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
v24.x.x
11.x.x
```

তোমার নিজের system-এ exact version আলাদা হতে পারে।

34.4 Project Initialize করা

একটি নতুন folder:

TERMINAL COMMAND

BASH

```
mkdir my-project
cd my-project
```

তারপর:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">init
```

এটি কিছু প্রশ্ন করবে। আর দ্রুত default configuration চাইলে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">init -y
```

এতে তৈরি হবে:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
package.json
```

34.5 package.json কী?

package.json হলো JavaScript/Node project-এর **project configuration এবং dependency manifest file**। Example:

PACKAGE MANIFEST (JSON)JSON

```
{
  "name": "my-project",
  "version": "1.0.0",
  "description": "My JavaScript project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {},
  "devDependencies": {}
}
```

এটি project-এর গুরুত্বপূর্ণ information ধরে রাখে।

Part 02 — Anatomy of package.json, Core Fields & Semantic Versioning (34.6 – 34.11)

34.6 package.json -এর গুরুত্বপূর্ণ fields

Field	কাজ
name	Project/package name
version	Project version
description	Project description
main	Entry point
scripts	Commands
dependencies	Production dependencies
devDependencies	Development dependencies
type	Module system
engines	Runtime version requirements

34.7 name

PACKAGE MANIFEST (JSON)JSON

```
{
  "name": "my-project"
}
```

এটি project/package-এর নাম। সাধারণত lowercase এবং npm naming rules অনুসরণ করা হয়।

34.8 version

PACKAGE MANIFEST (JSON)JSON

```
{
  "version": "1.0.0"
}
```

এটি project/package-এর current version। এখানেই আসে **Semantic Versioning**।

34.9 Semantic Versioning — SemVer

সাধারণ format:

SEMANTIC VERSIONING (SEMVER) SPECIFICATION

MAJOR.MINOR.PATCH

Example:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

2.5.3

মানে:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

2 → Major

5 → Minor

3 → Patch

Major

Breaking changes:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

1.0.0

↓

2.0.0

আগের code compatibility নষ্ট হতে পারে।

Minor

নতুন backward-compatible feature:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

1.2.0

↓

1.3.0

Patch

Bug fix:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

1.2.3

↓

1.2.4

34.10 Dependency কী?

Dependency হলো এমন package যার ওপর তোমার project নির্ভর করে। ধরো:

TERMINAL COMMAND

BASH

-weight:700;">npm -weight:600;">install express

তাহলে express project-এর dependency। package.json:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"dependencies": {
    600;">"express": "^5.1.0"
  }
}
```

34.11 Real-life Example

ধরো তুমি restaurant বানাচ্ছে। তোমার restaurant:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Restaurant

- Chef
- Waiter
- Kitchen
- Electricity

Electricity ছাড়া restaurant চলবে না। ঠিক তেমনি:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Your App

- React
- Express
- Database Driver
- Other Packages

এগুলো application-এর dependency হতে পারে।

Part 03 — Package Installation, node_modules & Lockfiles (34.12 – 34.16)

34.12 npm install

Package install:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install lodash
```

Short form:

TERMINAL COMMAND

BASH

```
-weight:700;">npm i lodash
```

এটি সাধারণত: 1. package download করে 2. node_modules তৈরি/আপডেট করে 3. package.json update করে 4. lockfile update করে

34.13 node_modules

Package install করার পর:

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
my-project/
├── node_modules/
├── package.json
├── package-lock.json
└── index.js
```

node_modules-এর মধ্যে installed packages থাকে। উদাহরণ:

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
node_modules/
├── lodash/
├── express/
└── ...
```

🔥 node_modules GitHub-এ commit করা উচিত? সাধারণত: **না**। .gitignore:

CONFIG FILE (GITIGNORE)

ENV

```
node_modules/
```

কারণ অন্য developer:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install
```

দিয়ে dependencies install করতে পারে।

34.14 package-lock.json

এটি dependency tree-এর **exact resolved versions** এবং related metadata lock করে রাখে। Example:

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
package.json
  ↓
Version range
  ↓
package-lock.json
  ↓
Exact dependency resolution
```

এটি team এবং CI/CD environment-এ consistent installation-এর জন্য গুরুত্বপূর্ণ।

34.15 package.json vs package-lock.json

package.json	package-lock.json
Project/dependency declaration	Resolved dependency tree lock
Version range থাকতে পারে	Exact resolved versions record করে
Human-edited	সাধারণত npm manage করে
Project configuration	Reproducible installation-এ সহায়্য করে

♦ GitHub-এ কোনটা রাখব?

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
package.json      ✓
package-lock.json ✓
node_modules      ✗
```

34.16 npm install বনাম npm ci

◆ npm install

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install
```

সাধারণত dependency install/update resolution করতে পারে এবং lockfile update করতে পারে।

◆ npm ci

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">ci
```

Clean, reproducible installation-এর জন্য ব্যবহৃত হয়, বিশেষ করে CI/CD-তে। এটি lockfile-এর ওপর বেশি strict।

◆ Real-life

📦 PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Developer Machine



npm install

CI/CD Server



npm ci

Part 04 — Dependencies vs DevDependencies & Package Lifecycle (34.17 – 34.22)

34.17 Dependency vs DevDependency

এটি খুব important।

📦 Dependency

Application চালানোর সময় দরকার:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install express
```

package.json:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"dependencies": {
    600;">"express": "^5.1.0"
  }
}
```

📦 DevDependency

Development-এর সময় দরকার:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install -D -weight:700;">eslint
```

অথবা:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install --save--weight:600;">dev -weight:700;">eslint
```

Result:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"devDependencies": {
    600;">"eslint": "^9.x"
  }
}
```

34.18 Example

ধরো React project:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
dependencies
├── react
└── react-dom

devDependencies
├── vite
├── eslint
└── prettier
```

সহজভাবে:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
Application Runtime
  ↓
dependencies

Development / Build / Testing
  ↓
devDependencies
```

⚠ Exact production bundling behavior tool/platform অনুযায়ী ভিন্ন হতে পারে, তাই শুধু category দেখে package runtime-এ থাকবে/থাকবে না এমন universal assumption করা ঠিক নয়।

34.19 Package Install-এর বিভিন্ন form

◆ Normal

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install axios
```

◆ Dev dependency

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install -D -weight:700;">eslint
```

◆ Specific version

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install lodash@4.17.21
```

◆ Latest

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install lodash@latest
```

◆ Global

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install -g some-package
```

Global install প্রয়োজন অনুযায়ী ব্যবহার করা হয়; project dependencies-এর জন্য সাধারণত local install preferred।

34.20 Package Uninstall

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">uninstall lodash
```

অথবা:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">remove lodash
```

34.21 Package Update

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">update
```

Specific package:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">update lodash
```

Installed packages দেখতে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm list
```

Top-level packages:

TERMINAL COMMAND

BASH

```
-weight:700;">npm list --depth=0
```

34.22 npm outdated

কোন package outdated তা দেখতে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">outdated
```

Example:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Package	Current	Wanted	Latest
react	18.x	18.x	19.x

Concept:

NPM DEPENDENCY & PROJECT ARCHITECTURE

Current → এখন installed
Wanted → **package.json**-এর allowed range-এর মধ্যে update
Latest → registry-এর latest version

Part 05 — NPM Registry, Package Discovery & Metadata Search (34.23 – 34.26)

34.23 NPM Registry

NPM package সাধারণত registry থেকে পাওয়া যায়। Concept:

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
Developer
↓
npm install react
↓
NPM Registry
↓
Package download
↓
node_modules
```

34.24 npm search

Package খুঁজতে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">search express
```

তবে modern development-এ official documentation, package repository এবং trusted sources দেখে package নির্বাচন করা ভালো।

34.25 npm info

Package সম্পর্কে information:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">info express
```

Specific version:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">info express version
```

34.26 npm view

Example:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">view react version
```

এটি registry থেকে package metadata দেখতে সাহায্য করে।

Part 06 — NPM Scripts Automation & Development Task Pipelines (34.27 – 34.30)

34.27 NPM Scripts

package.json:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"scripts": {
    600;">"start": "node index.js",
    600;">"dev": "node --watch index.js",
    600;">"test": "node test.js"
  }
}
```

Run:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">start
```

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">test
```

34.28 npm run কী করে?

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

মানে:

📦 NPM DEPENDENCY & PROJECT ARCHITECTURE

```
npm
↓
package.json
↓
scripts
↓
dev
↓
associated command
```

34.29 npm start

Special shorthand:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">start
```

এটি:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">start
```

এর equivalent। একইভাবে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">test
```

special script হিসেবে ব্যবহার করা যায়।

34.30 Practical Example — Scripts

package.json:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"scripts": {
    600;">"dev": "vite",
    600;">"build": "vite build",
    600;">"preview": "vite preview"
  }
}
```

Commands:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">build
```

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run preview
```

React/Vite project-এ এ ধরনের scripts খুব common।

34.31 npx কী?

npx package-এর executable command run করতে ব্যবহৃত হয়। Example:

TERMINAL COMMAND

BASH

```
-weight:700;">npx -weight:700;">vite
```

আরেকটি common pattern:

TERMINAL COMMAND

BASH

```
-weight:700;">npx -weight:600;">create--weight:700;">vite
```

এটি package command চালানোর জন্য ব্যবহৃত হয়।

♦ সহজ পার্থক্য

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

npm install



Package install/manage

npx



Package executable run

34.32 npx কি package install করে?

npx প্রয়োজন অনুযায়ী package executable resolve/download/cache করে run করতে পারে। তাই:

TERMINAL COMMAND

BASH

```
-weight:700;">npx some-tool
```

মানে সবসময় permanent local dependency হিসেবে project-এ install হয়ে গেছে—এমন নয়।

34.33 Vite কী?

Vite হলো modern frontend development/build tool। React project তৈরি:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">create -weight:700;">vite@latest
```

তারপর:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install
```

Run:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

34.34 Vite কী করে?

Development-এর সময়:

MODERN BUILD & BUNDLING PIPELINE

```
Your Code
  ↓
Vite Dev Server
  ↓
Browser
```

Production build:

MODERN BUILD & BUNDLING PIPELINE

```
Source Code
  ↓
Vite Build
  ↓
Optimized Assets
  ↓
dist/
```

34.35 Build কী?

Development source code production-এর জন্য optimize করে output তৈরি করাকে broadly build process বলা হয়। Example:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">build
```

Output:

MODERN BUILD & BUNDLING PIPELINE

```
dist/
├── index.html
├── assets/
│   ├── index-xxxx.js
│   └── index-xxxx.css
```

Part 08 — Build Pipelines, Tree Shaking, Minification & Source Maps (34.36 – 34.40)

34.36 Development vs Production

◆ Development

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

সাঁধারিণত:

- fast feedback
- hot reload/HMR
- debugging
- source-oriented workflow

◆ Production

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">build
```

সাঁধারিণত:

- optimized assets
- minification
- bundling/chunking
- deployment-ready output

34.37 Bundler কী?

একটি modern frontend application-এ অনেক module থাকতে পারে:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
main.js
├── utils.js
├── api.js
├── components.js
└── styles.css
```

Build tooling এগুলো process করে browser-এর জন্য optimized assets তৈরি করতে পারে। এখানে bundling/chunking/module transformation-এর মতো কাজ হতে পারে।

34.38 Tree Shaking

Unused code বাদ দেওয়ার optimization-কে tree shaking বলা হয়। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
export function add() {}

export function subtract() {}

export function multiply() {}
```

তুমি শুধু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
import { add } from "./math.js";
```

ব্যবহার করলে build tool বুঝতে পারে অন্য exports ব্যবহৃত হচ্ছে না এবং production output-এ সেগুলো বাদ দেওয়ার সুযোগ পেতে পারে। ⚠️ Tree shaking সবসময় perfect হয় না; static analyzability এবং package/module structure গুরুত্বপূর্ণ।

34.39 Minification

Code-এর unnecessary whitespace/comments ইত্যাদি কমিয়ে output ছোট করা হয়। Before:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {
  return a + b;
}
```

Production output conceptually এমন হতে পারে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a,b){return a+b}
```

এতে download size কমতে পারে।

34.40 Source Map

Production/minified code debug করতে source map সাহায্য করে। Concept:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
Original Code
  ↓
Build
  ↓
Minified Code
  +
Source Map
  ↓
DevTools
  ↓
Original source mapping
```

সাধারণত:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
.js
.js.map
```

দেখতে পাওয়া যেতে পারে।

Part 09 — Code Quality Engineering: ESLint, Prettier & Linter Synergy (34.41 – 34.44)

34.41 ESLint

ESLint JavaScript/TypeScript code-এর সমস্যা, ভুল pattern এবং style-related issues detect করতে সাহায্য করে। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const x = 10
```

তোমার ESLint configuration অনুযায়ী semicolon বা অন্যান্য rule নিয়ে warning/error দিতে পারে। Install:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install -D -weight:700;">eslint
```

তারপর project অনুযায়ী configuration setup করা হয়।

34.42 ESLint কেন?

CODE QUALITY & LINTER WORKFLOW

```
Code
  ↓
ESLint
  ↓
Problems / Warnings
  ↓
Fix
  ↓
Better Code
```

এটি:

- bugs detect করতে সাহায্য করে
- consistent code style maintain করতে সাহায্য করে
- team development সহজ করে

34.43 Prettier

Prettier হলো code formatter। Example: Before:

```
JAVASCRIPT SOURCE JAVASCRIPT
const user={name:"Ripon",age:24};
```

Prettier এটাকে format করতে পারে:

```
JAVASCRIPT SOURCE JAVASCRIPT
const user = {
  name: "Ripon",
  age: 24,
};
```

Install:

```
TERMINAL COMMAND BASH
-weight:700;">npm -weight:600;">install -D -weight:700;">prettier
```

34.44 ESLint vs Prettier

ESLint	Prettier
Code quality/linting	Formatting
Potential problems detect	Code format করে
Rules	Layout/style
Bug-prone patterns detect করতে পারে	Consistent appearance

দুটো একসাথে ব্যবহার করা খুব common।

Part 10 — Environment Variables, .env, .gitignore & Supply Chain Security (34.45 – 34.48)

34.45 Environment Variables

Environment-specific configuration রাখার জন্য environment variables ব্যবহার করা হয়। Example:

```
PACKAGE MANAGEMENT & TOOLING ECOSYSTEM
API_URL=https://api.example.com
```

Node.js-এ:

```
JAVASCRIPT SOURCE JAVASCRIPT
console.log(process.env.API_URL);
```

34.46 .env File

Example:

CONFIG FILE (ENV)

ENV

```
PORT=5000
DATABASE_URL= ...
API_URL=https://api.example.com
```

Node.js application-এ environment variables load করার জন্য runtime/tool অনুযায়ী ব্যবস্থা লাগে। # ⚠️ খুব গুরুত্বপূর্ণ Security Rule Frontend application-এ environment variable লিখলেই সেটা secret হয় না। যেমন:

CONFIG FILE (ENV)

ENV

```
VITE_API_KEY=123456
```

Vite-এর frontend build-এ expose করা variable browser-এর কাছে চলে যেতে পারে। তাই:

📦 PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Frontend

- ✗ Secret API key
- ✗ Database password
- ✗ Private credentials
- ✗ Server secret

Frontend-এ কখনো real secret রাখবে না।

34.47 .gitignore

Sensitive/local/generated files Git থেকে বাদ দিতে:

CONFIG FILE (GITIGNORE)

ENV

```
node_modules/
.env
dist/
```

কিন্তু project-specific rules অনুযায়ী .env.example রাখা যেতে পারে:

CONFIG FILE (ENV)

ENV

```
PORT=
DATABASE_URL=
API_URL=
```

এতে team বুঝতে পারে কোন variables প্রয়োজন।

34.48 Package Security

External package ব্যবহার মানেই security risk-এর possibility থাকে। Check:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">audit
```

Fix attempt:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">audit -weight:600;">fix
```

Package update:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">update
```

কিন্তু blindly:

TERMINAL COMMAND

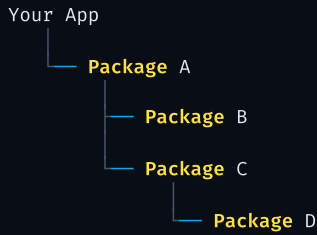
BASH

```
-weight:700;">npm -weight:600;">audit -weight:600;">fix --force
```

ব্যবহার করা উচিত নয়। কারণ breaking changes আসতে পারে।

34.49 Dependency Tree

একটি package-এর নিজের dependency থাকতে পারে।

 PACKAGE MANAGEMENT & TOOLING ECOSYSTEM


এজন্য node_modules অনেক বড় হতে পারে।

34.50 Direct vs Transitive Dependency

♦ Direct dependency

তুমি নিজে install করেছো:

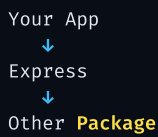
 TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install express
```

♦ Transitive dependency

Express-এর dependency হিসেবে অন্য package এসেছে।

 PACKAGE MANAGEMENT & TOOLING ECOSYSTEM
**34.51** Version Ranges

package.json-এ:

 PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"dependencies": {
    600;">"example": "^1.2.3"
  }
}
```

এখানে ^ version range বোঝায়। আর:

 PACKAGE MANIFEST (JSON)

JSON

```
600;">"example": "~1.2.3"
```

এর behavior আলাদা।

34.52 ^ এবং ~

সাধারণ SemVer context-এ:

♦ Caret

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

^1.2.3

সাধারণত compatible minor/patch updates allow করে, কিন্তু 2.0.0 নয়।

♦ Tilde

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

~1.2.3

সাধারণত patch-level updates-এর মধ্যে থাকে। ⚠️ Exact behavior package versioning rules এবং major version (0.x) অনুযায়ী nuance থাকতে পারে।

34.53 Exact Version

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"dependencies": {
    600;">"example": "1.2.3"
  }
}
```

এতে range না রেখে exact version declare করা হয়। কিন্তু reproducible installation-এর জন্য lockfile-ও গুরুত্বপূর্ণ।

34.54 Global vs Local Package

♦ Local

TERMINAL COMMAND

BASH

```
-weight:700;">"npm -weight:600;">"install -weight:700;">"eslint
```

থাকে:

PACKAGE MANAGEMENT & PROJECT ARCHITECTURE

project/node_modules/

♦ Global

TERMINAL COMMAND

BASH

```
-weight:700;">"npm -weight:600;">"install -g ...
```

Machine-wide command-এর জন্য।

♦ Best practice

Project-specific dependency:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

Local install

সাধারণত preferred।

34.55 npm exec

Package executable run করার আরেকটি modern command:

TERMINAL COMMAND

BASH

```
-weight:700;">"npm -weight:600;">"exec -- -weight:700;">"eslint .
```

অনেক ক্ষেত্রে npx-এর alternative হিসেবে ব্যবহার করা যায়।

34.56 Package Scripts + Arguments

Script:

```
PACKAGE MANIFEST (JSON)
{
  600;">"scripts": {
    600;">"lint": "eslint ."
  }
}
```

JSON

Run:

```
TERMINAL COMMAND
-weight:700;">npm -weight:600;">run lint
```

BASH

Additional arguments pass করতে:

```
TERMINAL COMMAND
-weight:700;">npm -weight:600;">run lint -- --weight:600;">fix
```

BASH

34.57 Practical Node Project

Structure:

 NPM DEPENDENCY & PROJECT ARCHITECTURE

```
my-node-app/
├── src/
│   └── index.js
├── .env
├── .gitignore
├── package.json
├── package-lock.json
└── README.md
```

package.json:

```
PACKAGE MANIFEST (JSON)
{
  600;">"name": "my-node-app",
  600;">"version": "1.0.0",
  600;">"type": "module",
  600;">"scripts": {
    600;">"start": "node src/index.js",
    600;">"dev": "node --watch src/index.js"
  }
}
```

JSON

Run:

```
TERMINAL COMMAND
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

BASH

34.58 Practical React/Vite Project

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
my-react-app/  
├── node_modules/  
├── public/  
├── src/  
│   ├── components/  
│   ├── App.jsx  
│   └── main.jsx  
├── .gitignore  
├── index.html  
├── package.json  
├── package-lock.json  
└── vite.config.js
```

Commands:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install
```

Development:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

Production build:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">build
```

Preview:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run preview
```

34.59 npm create vs npx

Modern tooling-এ ভূমি এমন command দেখতে পারো:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">create -weight:700;">vite@latest
```

এটি project scaffolding tool চালায়। অন্যদিকে:

TERMINAL COMMAND

BASH

```
-weight:700;">npx some-tool
```

সাধারণভাবে package executable run করার pattern।

34.60 Package Manager Alternatives

NPM ছাড়াও আছে:

♦ Yarn

TERMINAL COMMAND

BASH

```
-weight:700;">yarn
```

♦ pnpm

TERMINAL COMMAND

BASH

```
-weight:700;">pnpm -weight:600;">install
```

♦ Bun

TERMINAL COMMAND

BASH

```
-weight:700;">bun -weight:600;">install
```

এগুলো JavaScript ecosystem-এর package/runtime tooling ecosystem-এর অংশ। তবে একটি project-এ package manager consistent রাখা ভালো।

34.61 packageManager Field

কখনও project-এ:

PACKAGE MANIFEST (JSON)

JSON

```
{
  600;">"packageManager": "npm@11.x"
}
```

এর মতো metadata থাকতে পারে। এটি team-কে কোন package manager/version ব্যবহার করার expectation জানাতে সাহায্য করতে পারে।

34.62 NPM Cache

NPM downloaded package metadata/content-এর cache ব্যবহার করতে পারে। Cache সম্পর্কে:

TERMINAL COMMAND

BASH

```
-weight:700;">npm cache verify
```

এটি cache integrity check করতে সাহায্য করে। সাধারণ সমস্যা হলেই সরাসরি cache delete করা সবসময় প্রথম solution নয়।

34.63 Package Installation Workflow

একজন developer যখন GitHub থেকে project clone করে:

TERMINAL COMMAND

BASH

```
-weight:700;">git -weight:600;">clone <repo>
```

তারপর:

TERMINAL COMMAND

BASH

```
cd project
```

তারপর:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">install
```

তারপর:

TERMINAL COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

Flow:

NPM DEPENDENCY & PROJECT ARCHITECTURE

```
GitHub
↓
Clone
↓
package.json
↓
npm install
↓
node_modules
↓
npm run dev
↓
Application
```

34.64 Production Deployment Workflow

ধরো Vite application:

MODERN BUILD & BUNDLING PIPELINE

```
Source Code
↓
npm install
↓
npm run build
↓
dist/
↓
Hosting
```

Node backend:

PACKAGE MANAGEMENT & TOOLING ECOSYSTEM

```
Source
↓
npm ci
↓
Environment Variables
↓
Start command
↓
Server
```

34.65 Common Mistakes — প্যাকেজ ও টুলিংয়ের সাধারণ ভুলসমূহ

- ❖ ❌ **node_modules** GitHub-এ push করা

CONFIG FILE (GITIGNORE)

ENV

```
node_modules/
```

- ❖ ❌ **.env** public repository-তে push করা

CONFIG FILE (GITIGNORE)

ENV

```
.env
```

- ❖ ❌ **package** blindly update করা

Update-এর আগে breaking changes/changelog/documentation দেখা উচিত।

- ❖ ❌ **npm audit fix --force** blindly চালানো

এতে breaking upgrade হতে পারে।

- ❖ ❌ **সব package** global install করা

Project dependency local রাখা সাধারণত ভালো।

- ❖ ❌ **package-lock** delete করে দেওয়া

কারণ reproducible dependency installation-এর সুবিধা নষ্ট হতে পারে।

- ❖ ❌ **Unknown package** blindly install করা

আগে package:

- downloads
- repository
- maintainer
- documentation
- recent activity
- vulnerabilities

দেখা ভালো।

Part 13 — Must Know Tooling Pillars, Quick Cheat Sheet, 13 Practice Labs & Final Mental Map

34.66 🔥 MUST KNOW — জাভাস্ক্রিপ্ট টুলিংয়ের ১০টি প্রধান স্তম্ভ

প্রফেশনাল ডেভেলপার হিসেবে প্রজেক্ট পরিচালনা ও ডিপ্লয়মেন্টে এই ১০টি মূল স্তম্ভ সর্বদা মেনে চলতে হবে:

★ 1. Manifest & Dependencies

- **package.json**: প্রজেক্টের মেটাডাটা ও ডিপেনডেন্সি ম্যানিফেস্ট
- **package-lock.json**: ছবছ আইডেন্টিক্যাল সাব-ডিপেনডেন্সি ট্রি লক
- **node_modules**: লোকাল ইনস্টল ফোল্ডার (কখনো গিটে কমিট করবেন না)
- **dependencies vs dev**: রানটাইম লাইব্রেরি বনাম ডেভেলপমেন্ট টুলস

★ 2. Versioning & Automation

- **SemVer (MAJOR.MINOR.PATCH)**: ব্রেকিং চেঞ্জ ও বাগফিক্স নিয়ম
- **npm ci**: সিআই/সিডি সার্ভারে ক্লিন ও অপরিবর্তনীয় ডিপেনডেন্সি ইনস্টলেশন
- **npm scripts**: dev, build, lint এবং test টাস্ক অটোমেশন
- **npm Runner**: গ্লোবাল ইনস্টল ছাড়া তাৎক্ষণিক প্যাকেজ এক্সিকিউট

★ 3. Build Tooling & Quality

- **Vite / Rollup**: ইলেকট্রন-স্পিড নেটিভ ESM ডেভেলপমেন্ট সার্ভার
- **Tree Shaking & Minify**: ডেড-কোড বাদ দিয়ে প্রোডাকশন বাউল সংকোচন
- **ESLint & Prettier**: কোড স্ট্যান্ডার্ড ও ফরম্যাটিং স্বয়ংক্রিয়করণ
- **.env Security**: সিক্রেট ও এপিআই কি গিট ট্র্যাকিং থেকে সুরক্ষিত রাখা

Command / Concept	Syntax / Option	Main Purpose & Mechanism
npm init -y	npm init -y	ডিফল্ট কনফিগারেশনে তাৎক্ষণিক package.json তৈরি
npm install	npm i <pkg>	রানটাইম ডিপেনডেন্সি ইনস্টল ও package.json-এ যুক্ত
npm install -D	npm i -D <pkg>	শুধু ডেভেলপমেন্টে প্রয়োজনীয় প্যাকেজ ইনস্টল (devDependencies)
npm install -g	npm i -g <pkg>	সিস্টেম-ওয়াইড গ্লোবাল সিএলআই টুল ইনস্টল
npm ci	npm ci	লকফাইল অনুযায়ী কঠোর ও ক্লিন ইনস্টলেশন (CI/CD-তে বাধ্যতামূলক)
npm run	npm run <script>	package.json-এ সংজ্ঞায়িত কাস্টম স্ক্রিপ্ট এক্সিকিউট
npm x	npm x <package>	প্যাকেজ লোকালি ইনস্টল না করে রিমোট থেকে সরাসরি রান
npm outdated	npm outdated	প্রজেক্টের কোন কোন প্যাকেজ পুরনো হয়েছে তা অডিট
npm update	npm update	SemVer রেঞ্জ মেনে অনুমোদিত মাইনর ও প্যাচ ভার্সন আপডেট
npm uninstall	npm un <pkg>	প্যাকেজ ও তার রেফারেন্স সম্পূর্ণ মুছে ফেলা
npm audit	npm audit	ডিপেনডেন্সি ট্রিতে পরিচিত সিকিউরিটি দুর্বলতা স্ক্যান
npm audit fix	npm audit fix	অটোমেটিক সিকিউরিটি প্যাচ ইনস্টল করে নিরাপত্তা নিশ্চিত
Caret (^)	"^1.2.3"	MAJOR অপরিবর্তিত রেখে MINOR ও PATCH অটো আপডেট
Tilde (~)	"~1.2.3"	MAJOR ও MINOR অপরিবর্তিত রেখে শুধু PATCH অটো আপডেট
Exact	"1.2.3"	কোনো অটো আপডেট হবে না, ছবছ এই ভার্সন থাকবে
Vite Dev	vite/npm run dev	নো-বান্ডলিং নেটিভ ESM দিয়ে ইন্সট্যান্ট ডেভ সার্ভার বুট
Vite Build	vite build	Rollup ও esbuild দিয়ে অস্টিমাইজড প্রোডাকশন বান্ডলিং
ESLint	npx eslint .	জাভাস্ক্রিপ্ট সিনট্যাক্স ভুল ও অ্যান্টি-প্যাটার্ন শনাক্ত
Prettier	npx prettier --write .	কোড ফরম্যাটিং ও স্পেসিং স্বয়ংক্রিয়ভাবে সাজানো
.gitignore	node_modules/, .env	সংবেদনশীল ও হেভি ফাইল গিট ট্র্যাকিং থেকে বাদ দেওয়া

Part 13.1 — 13 Hands-On Practice Labs with Complete Production Solutions

Lab #1 [BEGINNER] — Project Initialization with npm init -y

সমস্যা ও লক্ষ্য: একটি নতুন NPM project তৈরি করুন (npm init -y) এবং স্বয়ংক্রিয়ভাবে তৈরি হওয়া package.json-এর স্ট্রাকচার বিশ্লেষণ করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# টার্মিনালে নতুন ফোল্ডার তৈরি করে প্রজেক্ট ইনিশিয়লাইজেশন
mkdir my-js-project
cd my-js-project
-weight:700;">npm -weight:600;">init -y
```

CONFIGURATION & IMPLEMENTATION (LAB #1)

JSON

```
{
  600;">"name": "my-js-project",
  600;">"version": "1.0.0",
  600;">"description": "Enterprise JavaScript Package",
  600;">"main": "src/index.js",
  600;">"type": "module",
  600;">"scripts": {
    600;">"test": "echo \"Error: no test specified\" && exit 1"
  },
  600;">"keywords": ["javascript", "tooling", "npm"],
  600;">"author": "FullStack Engineer",
  600;">"license": "MIT"
}
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** npm init -y কোনো প্রশ্ন ছাড়াই তাৎক্ষণিক একটি স্ট্যান্ডার্ড package.json তৈরি করে। আধুনিক প্রজেক্টে ES Module ইমপোর্ট ব্যবহারের জন্য "type": "module" যোগ করা অত্যন্ত গুরুত্বপূর্ণ।

Lab #2 [BEGINNER] — Runtime Dependency Installation & ESM Usage

সমস্যা ও লক্ষ্য: একটি থার্ড-পার্টি লাইব্রেরি (lodash-es) ইনস্টল করুন এবং মডার্ন জাভাস্ক্রিপ্ট ফাইলে ইমপোর্ট করে অ্যারে অপারেশন পরিচালনা করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# টার্মিনালে lodash-es ইনস্টল
-weight:700;">npm -weight:600;">install lodash-es
```

CONFIGURATION & IMPLEMENTATION (LAB #2)

JAVASCRIPT

```
// src/index.js
import { chunk, shuffle, uniq } from 'lodash-es';

const rawScores = [10, 20, 30, 20, 40, 50, 10, 60, 70, 80];

// ডুপ্লিকেট সরানো
const uniqueScores = uniq(rawScores);
console.log('Unique Scores:', uniqueScores);

// ৩টি করে চাঞ্জে বিভক্ত করা
const scoreChunks = chunk(uniqueScores, 3);
console.log('Score Chunks:', scoreChunks);

// অ্যারে এলোমেলো করা
console.log('Shuffled:', shuffle(uniqueScores));
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** npm install রানটাইম লাইব্রেরিগুলোকে dependencies ফিল্ডে যুক্ত করে যা ব্রাউজার বা প্রোডাকশন সার্ভারে একচুয়াল কোড রান করতে প্রয়োজন হয়।

Lab #3 [BEGINNER] — DevDependency Installation with -D Flag

সমস্যা ও লক্ষ্য: একটি ডেভেলপমেন্ট-অনলি প্যাকেজ (eslint) ইনস্টল করুন এবং devDependencies-এর তাৎপর্য বুঝুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# ডেভেলপমেন্ট টুল হিসেবে ESLint ইনস্টল
-weight:700;">npm -weight:600;">install -D -weight:700;">eslint
```

CONFIGURATION & IMPLEMENTATION (LAB #3)

JSON

```
{
  600;">"name": "my-js-project",
  600;">"version": "1.0.0",
  600;">"type": "module",
  600;">"dependencies": {
    600;">"lodash-es": "^4.17.21"
  },
  600;">"devDependencies": {
    600;">"eslint": "^9.0.0"
  }
}
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** -D বা --save-dev ফ্ল্যাগ নির্দেশ করে যে এই টুলটি কেবল কোড লেখা, লিন্টিং বা টেস্টিংয়ের সময় প্রয়োজন; প্রোডাকশন বিল্ডে বা ক্লায়েন্টের কাছে এটি শিপ করার প্রয়োজন নেই।

Lab #4 [BEGINNER] — Custom NPM Scripts Creation (dev, build, start)

সমস্যা ও লক্ষ্য: package.json-এ আধুনিক স্ক্রিপ্ট পাইপলাইন কনফিগার করুন এবং টার্মিনাল থেকে npm run দিয়ে এক্সিকিউট করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# স্ক্রিপ্ট রান করার কমান্ড
-weight:700;">npm -weight:600;">run -weight:600;">dev      # ওয়াচ মোডে লাইভ রান
-weight:700;">npm -weight:600;">run -weight:600;">build    # প্রোডাকশন বিন্ড ও অ্যাসেট তৈরি
-weight:700;">npm -weight:600;">start                    # প্রোডাকশন সার্ভার স্টার্ট
```

CONFIGURATION & IMPLEMENTATION (LAB #4)

JSON

```
{
  600;">"scripts": {
    600;">"dev": "node --watch src/index.js",
    600;">"build": "echo 'Building production bundle ... ' && mkdir -p dist && cp -r src/* dist/",
    600;">"start": "node dist/index.js",
    600;">"lint": "eslint src/**/*.js"
  }
}
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** npm scripts টার্মিনালে দীর্ঘ ও জটিল কমান্ড বারবার টাইপ করার ঝামেলা দূর করে এবং দলের সকল ডেভেলপারের জন্য একীভূত ও স্ট্যান্ডার্ডাইজড কমান্ড ইন্টারফেস তৈরি করে।

Lab #5 [BEGINNER] — Critical .gitignore Configuration for Node Projects

সমস্যা ও লক্ষ্য: node_modules এবং .env ফাইল যেন কখনো গিট রিপোজিটরিতে পুশ না হয় তার জন্য প্রোডাকশন গ্রেড .gitignore ফাইল তৈরি করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# গিট স্ট্যাটাস চেক করে ইগনোর হওয়া নিশ্চিত করা
-weight:700;">git status
```

CONFIGURATION & IMPLEMENTATION (LAB #5)

ENV

```
# Dependencies (কখনো গিটে পুশ করবেন না, হাজার হাজার ফাইল থাকে)
node_modules/
/.pnp
.pnp.js

# Environment Variables & Secrets (সিকিউরিটি রিস্ক)
.env
.env.local
.env.production

# Build Outputs & Cache
dist/
build/
.npm/
.eslintcache

# Operating System Files
.DS_Store
Thumbs.db
npm-debug.log*
yarn-debug.log*
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** node_modules গিটে পুশ করলে রিপোজিটরি সাইজ শত শত মেগাবাইট হয়ে যায়। আর .env পুশ করলে পাসওয়ার্ড বা এপিআই কি ইন্টারনেটে লিক হয়ে মারাত্মক সিকিউরিটি ব্রিচ ঘটে।

Lab #6 [INTERMEDIATE] — Clean Node.js Application Architecture with Watch Mode

সমস্যা ও লক্ষ্য: একটি স্ট্যান্ডার্ড Node.js প্রজেক্ট ফোল্ডার স্ট্রাকচার তৈরি করুন এবং Node 20+ এর নেটিভ `--watch` ফ্ল্যাগ দিয়ে হট-রিলোড ডেভ সার্ভার চালান।

TERMINAL EXECUTION COMMANDS

BASH

```
# প্রজেক্ট রান
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

CONFIGURATION & IMPLEMENTATION (LAB #6)

JAVASCRIPT

```
// src/index.js
import http from 'node:http';

const PORT = process.env.PORT || 3000;

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({
    status: 'online',
    runtime: 'Node.js LTS',
    timestamp: new Date().toISOString()
  }));
});

server.listen(PORT, () => {
  console.log(`🔥 Server active at http://localhost:${PORT}`);
  console.log('⚡ Watching src/ for instant code changes ...');
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** নোডজেএস ভার্সন ১৮.১১+ থেকে নোডমনের (nodemon) কোনো প্রয়োজন নেই; নেটিভ `--watch` ফ্ল্যাগ দিয়ে কোড এডিট করার সাথে সাথে নোড স্বয়ংক্রিয়ভাবে রিস্টার্ট হয়।

Lab #7 [INTERMEDIATE] — Modern Frontend Scaffolding with Vite

সমস্যা ও লক্ষ্য: `npm create vite@latest` দিয়ে একটি আধুনিক, হাই-পারফরম্যান্স ভ্যানিলা বা রিঅ্যাক্টিভ ফ্রন্টএন্ড প্রজেক্ট তৈরি করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# ১. Vite দিয়ে প্রজেক্ট স্কাফোল্ড করা
-weight:700;">npm -weight:600;">create -weight:700;">vite@latest enterprise-frontend -- --template vanilla

# ২. ডিরেক্টরিতে প্রবেশ করে ডিপেনডেন্সি ইনস্টল
cd enterprise-frontend
-weight:700;">npm -weight:600;">install

# ৩. লাইভ ডেভ সার্ভার বুট
-weight:700;">npm -weight:600;">run -weight:600;">dev
```

CONFIGURATION & IMPLEMENTATION (LAB #7)

JAVASCRIPT

```
// vite.config.js (Production Ready Configuration)
import { defineConfig } from 'vite';

export default defineConfig({
  server: {
    port: 5173,
    open: true
  },
  build: {
    outDir: 'dist',
    minify: 'esbuild',
    sourcemap: false
  }
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** Vite ডেভেলপমেন্ট টাইমে কোনো বাউন্ডিং করে না; ব্রাউজারের নেটিভ ES Modules ব্যবহার করে মিলিসেকেন্ডে ডেভ সার্ভার লোড করে, যা ওয়েবপ্যাকেজের তুলনায় ৫০ গুণ দ্রুত।

Lab #8 [INTERMEDIATE] — Production Build Pipeline & Asset Inspection

সমস্যা ও লক্ষ্য: একটি প্রোডাকশন বিল্ড তৈরি করুন (npm run build) এবং dist/ ফোল্ডারের মিনিফাইড জাভাস্ক্রিপ্ট ও হ্যাশড অ্যাসেট পরীক্ষা করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# বিল্ড কমান্ড রান
-weight:700;">npm -weight:600;">run -weight:600;">build

# বিল্ড আউটপুট প্রিভিউ
-weight:700;">npm -weight:600;">run preview
```

CONFIGURATION & IMPLEMENTATION (LAB #8)

HTML

```
⚡— dist/index.html (মিনিফাইড প্রোডাকশন আর্কিটেকচার) —>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" crossorigin href="/assets/index-D7h2k9La.css">
  <script type="module" crossorigin src="/assets/index-Bq1x90Lm.js"></script>
</head>
<body>
  <div id="app"></div>
</body>
</html>
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** বিল্ডের সময় Rollup অপ্টিমাইজেশন কোড মুছে ফেলে (Tree Shaking), ভ্যারিয়েবলের নাম ছোট করে (Minification) এবং ক্যাশিংয়ের জন্য ফাইলের নামের সাথে হ্যাশ (যেমন: index-Bq1x90Lm.js) যোগ করে।

Lab #9 [INTERMEDIATE] — Deep Comparative Matrix: npm install vs npm ci vs npm update

সমস্যা ও লক্ষ্য: npm install, npm ci এবং npm update-এর প্রয়োগের পার্থক্য, কাজের মেকানিজম এবং কখন কোনটি ব্যবহার করতে হবে তা তুলনামূলক চাটে উপস্থাপন করুন।

TERMINAL EXECUTION COMMANDS

BASH

```
# প্রজেক্ট ও সিআই পাইপলাইনে সঠিক কমান্ড নির্বাচন
-weight:700;">npm -weight:600;">install      # নতুন প্যাকেজ যোগ বা লোকাল ডেভেলপমেন্ট শুরুর সময়
-weight:700;">npm -weight:600;">ci          # GitHub Actions বা Jenkins সিআই/পিডি বিল্ডে ১০০% নির্ভরযোগ্য
-weight:700;">npm -weight:600;">update      # প্যাকেজগুলোর অনুমোদিত লেটেস্ট মাইনর/প্যাচ আপগ্রেড করতে
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** npm install লকফাইল পরিবর্তন করতে পারে, কিন্তু npm ci কখনো লকফাইল স্পর্শ করে না; লকফাইলে কোনো অসঙ্গতি থাকলে npm ci সজে সজে এর দিয়ে বিল্ড থামিয়ে দেয়।

Lab #10 [ADVANCED] — Full-Stack Multi-Package Monorepo Architecture

সমস্যা ও লক্ষ্য: frontend/ (Vite) এবং backend/ (Express) আলাদা package.json ও লকফাইল দিয়ে ম্যানেজ করার একটি স্বয়ংসম্পূর্ণ প্রোডাকশন আর্কিটেকচার।

TERMINAL EXECUTION COMMANDS

BASH

```
# রুট লেভেল থেকে সব-প্রজেক্ট চালানোর কমান্ড
-weight:700;">npm -weight:600;">run -weight:600;">dev:frontend  # ফ্রন্টএন্ড স্টার্ট
-weight:700;">npm -weight:600;">run -weight:600;">dev:backend   # ব্যাকএন্ড স্টার্ট
```

CONFIGURATION & IMPLEMENTATION (LAB #10)

JSON

```
{
  600;">"name": "fullstack-platform-root",
  600;">"private": 600;">true,
  600;">"scripts": {
    "install:all": "npm --prefix backend install && npm --prefix frontend install",
    "dev:backend": "npm --prefix backend run dev",
    "dev:frontend": "npm --prefix frontend run dev",
    "build:frontend": "npm --prefix frontend run build"
  }
}
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** মোনোরিপো স্ট্রাকচারে ক্লায়েন্ট ও সার্ভারের কোড একই রিপোজিটরিতে আলাদা মডিউলে থাকে, ফলে একই সাথে সম্পূর্ণ ফুলস্ট্যাক প্রজেক্ট গিটহাবে মাইনটেইন করা সহজ হয়।

Lab #11 [ADVANCED] — Automated Code Quality Pipeline: ESLint + Prettier Synergy

সমস্যা ও লক্ষ্য: প্রজেক্টে ESLint ও Prettier কনফিগার করুন যাতে কোড সেভ করার সাথে সাথে এরর ধরা পড়ে এবং অটো-ফরম্যাটিং সম্পন্ন হয়।

TERMINAL EXECUTION COMMANDS

BASH

```
# লিস্টার ও ফরম্যাটার ডিপেনডেন্সি ইনস্টল
-weight:700;">npm -weight:600;">install -D -weight:700;">eslint -weight:700;">prettier -weight:700;">eslint-config--
weight:700;">prettier
```

CONFIGURATION & IMPLEMENTATION (LAB #11)

JAVASCRIPT

```
// .eslintrc.json
{
  "env": {
    "browser": true,
    "es2024": true,
    "node": true
  },
  "extends": ["eslint:recommended", "prettier"],
  "parserOptions": {
    "ecmaVersion": "latest",
    "sourceType": "module"
  },
  "rules": {
    "no-unused-vars": "warn",
    "no-console": "off",
    "prefer-const": "error"
  }
}

// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

প্রোডাকশন অন্তর্দৃষ্টি: ESLint মূলত কোডের কোয়ালিটি (বাগ, অব্যবহৃত ভ্যারিয়েবল) চেক করে আর Prettier কোডের স্টাইল ও বিউটিফিকেশন (ইন্ডেন্টেশন, কোটেশন) নিয়ন্ত্রণ করে।

Lab #12 [ADVANCED] — Secure Environment Variables Management with dotenv

সমস্যা ও লক্ষ্য: ডেভেলপমেন্ট ও প্রোডাকশনে পোর্ট, ডেটাবেস স্থিতি এবং এপিআই কি নিরাপদে হ্যান্ডেল করার প্রোডাকশন আর্কিটেকচার।

TERMINAL EXECUTION COMMANDS

BASH

```
# dotenv লাইব্রেরি ইনস্টল
-weight:700;">npm -weight:600;">install dotenv
```

CONFIGURATION & IMPLEMENTATION (LAB #12)

JAVASCRIPT

```
// .env (Secret Credentials - Git Ignored)
PORT=5000
DATABASE_URL="postgresql://admin:superSecretPassword@localhost:5432/production_db"
JWT_SECRET="d83f98a21f7c8e9b4a123456789abcdef"

// .env.example (Public Template - Committed to Git)
PORT=5000
DATABASE_URL="postgresql://user:password@localhost:5432/dbname"
JWT_SECRET="your_jwt_secret_key_here"

// src/config.js (ESM Environment Loader)
import 'dotenv/config';

export const config = {
  port: process.env.PORT || 3000,
  dbUrl: process.env.DATABASE_URL,
  jwtSecret: process.env.JWT_SECRET
};

if (!config.jwtSecret) {
  throw new Error('FATAL: JWT_SECRET environment variable is missing!');
}
```

🔑 **প্রোডাকশন অন্তর্দৃষ্টি:** .env ফাইল সবসময় গিট ইগনোর থাকবে এবং রিপোজিটরিতে কেবল .env.example কমিট করতে হবে যাতে অন্য ডেভেলপাররা প্রয়োজনীয় ভ্যারিয়েবলের তালিকা দেখতে পায়।

Lab #13 [ADVANCED] — Supply Chain Security Audit & Vulnerability Remediation

সমস্যা ও লক্ষ্য: npm audit চালিয়ে ডিপেনডেন্সির সিকিউরিটি ত্রুটি পরীক্ষা, সিভিই (CVE) স্কোর বিশ্লেষণ এবং নিরাপদে ফিক্স করার বাস্তব কমান্ড।

TERMINAL EXECUTION COMMANDS

BASH

```
# ১. ডিপেনডেন্সি সিকিউরিটি স্ক্যান
-weight:700;">npm -weight:600;">audit

# ২. সেমিকম্প্যাটিবল অটো-ফিক্স
-weight:700;">npm -weight:600;">audit -weight:600;">fix

# ৩. ব্রেকিং চেঞ্জ রিভিউসহ ফোর্স ফিক্স (সতর্কতার সাথে)
-weight:700;">npm -weight:600;">audit -weight:600;">fix --force

# ৪. প্রোডাকশন অনলি অডিট (devDependencies বাদ দিয়ে)
-weight:700;">npm -weight:600;">audit --omit=-weight:600;">dev
```

CONFIGURATION & IMPLEMENTATION (LAB #13)

ENV

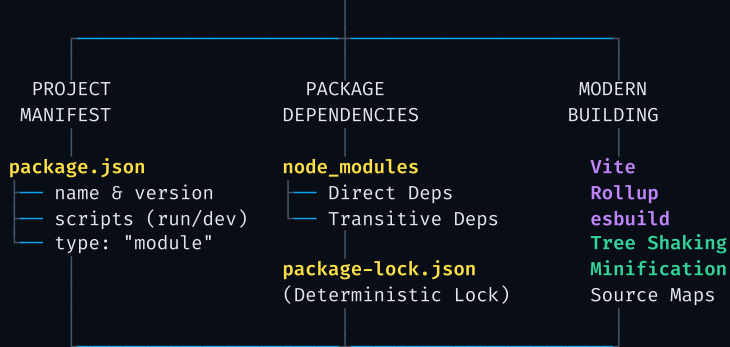
```
# NPM Security Audit Summary
found 0 vulnerabilities (Dependencies Verified Safe ✅)
# যদি কোনো হাই/ক্রিটিক্যাল রিস্ক পাওয়া যায়:
# 1 High Severity Vulnerability found in 'cross-spawn'
# Fixed by upgrading package to ≥7.0.5
```

🔑 **প্রোডাকশন অন্তর্দৃষ্টি:** সফটওয়্যার সাপ্লাই চেইন অ্যাটাক (যেমন ক্ষতিকর ম্যালওয়্যার লাইব্রেরি ইনজেকশন) থেকে সুরক্ষা নিশ্চিত করতে নিয়মিত npm audit চালানো বাধ্যতামূলক।

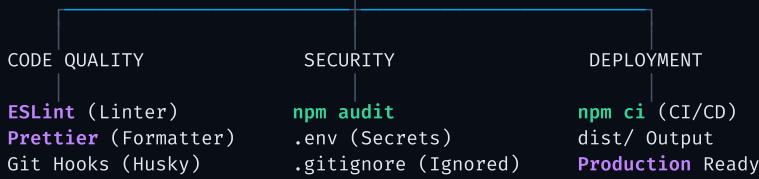
পুরো Chapter 34-এর টুলিং, প্যাকেজ ও আর্কিটেকচারাল পাইপলাইন একসাথে:

📦 FULL TOOLING & NPM ARCHITECTURE MAP

JAVASCRIPT TOOLING ECOSYSTEM



DEVELOPER WORKFLOW & PIPELINE



👤 এক লাইনে Chapter 34: NPM হলো জাভাস্ক্রিপ্ট প্রজেক্টের প্যাকেজ ও ডিপেনডেন্সি ম্যানেজমেন্ট সিস্টেম; আর package.json, lockfile, scripts, Vite এবং কোয়ালিটি টুলস (ESLint/Prettier) মিলে একটি মডার্ন জাভাস্ক্রিপ্ট প্রজেক্টের সম্পূর্ণ লাইফসাইকেল ও প্রডাকশন পাইপলাইন পরিচালনা করে।

📖 Curriculum Progression: Chapter 33 → 34 → 35

📦 ROADMAP CONTINUITY PIPELINE

Chapter 33: Web Performance & Optimization



অ্যাপ্লিকেশনের লোড টাইম, রেন্ডারিং স্পিড ও কোর ওয়েব ভাইটালস অপটিমাইজেশন



Chapter 34: NPM & Modern JavaScript Tooling (THIS CHAPTER)



প্যাকেজ ম্যানেজমেন্ট, প্রজেক্ট স্ক্যাফোল্ডিং, বান্ডলিং (Vite) ও ওয়ার্কফ্লো অটোমেশন



Chapter 35: Testing JavaScript



Unit Testing, Integration Testing, Assertions, Vitest/Jest, Mocking & TDD